

Reviewer Pack — Minimal Rank of Quantum Entropy over XOR-AND Tree Width

Entry e8d775b04736 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 22:08:10 UTC

Minimal Rank of Quantum Entropy over XOR-AND Tree Width

Entry ID: e8d775b04736 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 22:08:10 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Quantum Entropy) **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

[‘For any read-twice branching program P with size n , the quantum entropy of P , denoted as $Q(P)$, is upper bounded by $O(\log n)$.’, ‘Furthermore, for any polynomially-size XOR-AND tree T representing a language in NP, there exists an injective mapping from the set of leaves of T to the set of states of a quantum system such that the quantum entropy of this system, denoted as $Q(S)$, is at least $\Omega(n)$.’, ‘Finally, no read-twice branching program P can have both $Q(P) = O(\log n)$ and the corresponding XOR-AND tree T with $|T| > 2^n$.’]

Rationale (proposer’s reasoning):

[‘Quantum entropy provides a measure of the quantum state complexity, which has been studied in quantum information theory for its fundamental properties. Applying this concept to complexity theory could potentially uncover new structural insights into the complexity of computational problems.’, ‘XOR-AND trees are known to be a powerful representation for NP-complete problems. By relating quantum entropy to XOR-AND tree width, we aim to explore the connection between quantum information processing and classical computation.’, ‘This conjecture bridges quantum theory with complexity theory by proposing a quantitative relationship that could distinguish between simple and complex computational tasks.’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a67bebad73542b31

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Quantum entropy $Q(P)$ is within $O(\log n)$ of program size n for all generated read-twice branching programs P up to $n=40$, AND Spearman rank correlation coefficient between $Q(P)$ and XOR-AND tree width $|T|$ over 30 seeds is ≥ 0.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quantum entropy" AND "XOR-AND tree width" - "minimal rank" AND quantum information theory AND complexity theory - "read-twice branching program" AND "injective mapping" AND NP

Top relevant hits considered: - [s2:2205.05713] Concise tensors of minimal border rank - [s2:10.1109/ACCESS.2023.3301571] 3-D Beamforming Training Using Mutually Unbiased Bases for Cell Discovery in mm-Wave Systems

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def quantum_entropy(n):
        return n * math.log2(n + 1)

    def xor_and_tree_width(n):
        if n == 0:
            return 0
        return 1 + max(xor_and_tree_width(n // 2), xor_and_tree_width(n - n // 2))

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        P = [random.randint(0, 1) for _ in range(n)]
        Q_P = quantum_entropy(n)
        T_width = xor_and_tree_width(len(P))

        results.append({
            "n": n,
            "Q_P": Q_P,
            "T_width": T_width
```

```

    })

    metric_value = sum(result["Q_P"] for result in results) / len(results)
    instances_tested = len(results)
    conjecture_holds = all(result["Q_P"] <= math.log2(result["n"]) + 1 for result in results)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Quantum Entropy",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ef7f6eb.py", line 61, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ef7f6eb.py", line 35, in run_trial
    T_width = xor_and_tree_width(len(P))
             ^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ef7f6eb.py", line 27, in xor_and_tree_width
    return 1 + max(xor_and_tree_width(n // 2), xor_and_tree_width(n - n // 2))
                  ^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0ef7f6eb.py", line 27, in xor_and_tree_width
    return 1 + max(xor_and_tree_width(n // 2), xor_and_tree_width(n - n // 2))

```

